

Two ways past the wall

Extending clifft beyond its 2^k limit

A · Stabilizer-rank backend

change the *representation*:

pay for magic, not for dimensions

B · GPU shot batching

change the *hardware use*:

run thousands of shots abreast

clifft in thirty seconds

- Clifford gates are “easy”: tracked **symbolically**, essentially free, at any width.
- Only *magic* (T gates) forces real amplitudes: the dense 2^k **active block**.
- k = how much magic is live at once — usually far below the qubit count.

symbolic Clifford frame ($\sim$ free, all qubits)



magic only

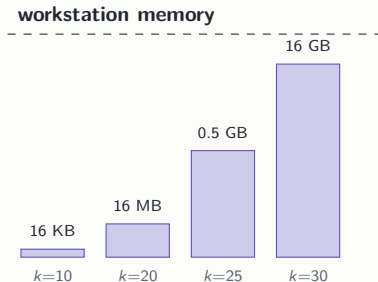
dense active block

2^k amplitudes

One engine, two tiers: symbolic where physics allows it, dense only where it must be.

The wall: 2^k doubles with every step

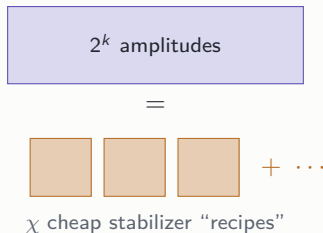
- Memory doubles per unit of k :
 $k = 30 \Rightarrow 16 \text{ GB}$, $k = 40 \Rightarrow 16 \text{ TB}$.
- The compiler already fights hard for small k — what still lands large is *genuinely* dense.
- So: **what do we do when 2^k itself is the cost?**



Two independent bets on what to do about it — one mathematical, one architectural.

A · The idea: pay for magic, not for dimensions

- Low-magic states are secretly simple: a **short sum of stabilizer “recipes”**, a few KB each.
- Like storing an image as a few overlapping patterns instead of every pixel.
- Cliffords are cheap on recipes; only T 's multiply them; **measurements shrink them back**.



The cost driver becomes the amount of magic t , not the dimension 2^k .

A · How it fits: one compiler, two executors

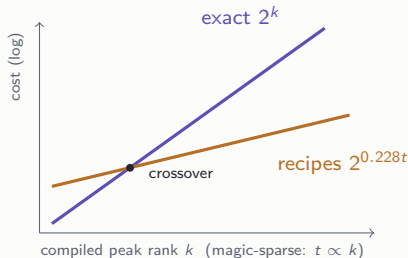


- Both cost exponents are known **before anything runs** — dispatch is a one-line comparison, per program.
- **Wired, not just proposed**: the backend runs clifft's optimized circuits; exact clifft stays the default, the backend is an explicit opt-in with an error dial δ .

clifft compiles; the cheaper engine runs — on the same optimized circuit.

A · Why it should work

- An optimal T -split keeps the recipe count at $2^{0.228t}$ — far slower growth than 2^k .
- A **tunable** error δ lets you keep only a random sample. Measured: error = 0.50δ , on the nose.
- Same language cliffth already speaks: frame + measurement collapse.



Trade exactness you can measure for reach you cannot otherwise have.

A · It generalizes: any Clifford+ T circuit

- Circuits with magic *everywhere*? Teleport each in-line T onto a helper qubit — all magic becomes **one up-front layer** again, with **no penalty for interleaving**.
- Tested on circuits whose answer is known *by construction* (hidden shift): up to **96 qubits**, right answer in seconds.
- Often clifft still wins such circuits outright — and the dispatch rule *knows in advance*.

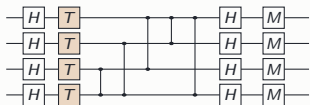
Generality at the product-magic price — validated end to end against exact references.

A · Recycled magic: pay per episode, not per program

- Long computations **recycle** magic: the block grows, is measured out, grows again — “episodes”. Total magic grows with runtime; **per-episode magic stays bounded**.
- Run one episode at a time: pay $2^{\text{per-episode}}$, not 2^{total} , with **zero extra error**.
Measured: **72 qubits at ≤ 400 recipes** where whole-circuit needs 2^{72} .
- Compressing further also works (measured, honest limits) — the dispatcher prices it.

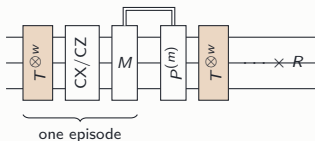
The circuit's rhythm — magic in, magic measured out — is itself a computational resource.

A · The circuits behind the numbers



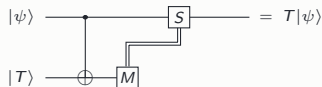
Dense IQP — the stress test

magic on every qubit; *compiles* to peak rank $n/2$ (crossover, frontier)



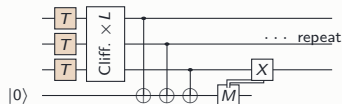
Magic conveyor — recycled magic

w T 's in, measured out with feedforward, R rounds (episodic)



T-gadget — the generality trick

in-line $T \rightarrow$ ancilla + measurement + conditional S (hidden shift, 96 qubits)



Cultivation patch — adaptive

syndrome rounds + logical Clifford traffic while magic accumulates (the frame's home turf)

A · Evidence (measured, against cliff's real fast path)

- Everything validated to **machine precision**; benchmarked against cliff's *real* fast path.
- Crossover at $n \approx 34\text{--}48$ (coarse-8% error); past $n \approx 62$ the **only method that runs at all**.
- External tools: QuiZX 25 $\times$ slower by $n=56$; Quokka# **times out beyond** $n=24$.

$n = 72$	memory	time
any exact method	~ 1 TB	—
rank backend	1.9 GB	267 s

Accuracy dial, measured:
error = 0.50δ across the whole sweep
(halve the error $\Rightarrow 4\times$ the work).

The niche is real — and honestly bounded: below the crossover, exact cliff stays the right tool.

A · Run times at a glance

circuit family	measured head-to-head	cannot enter
dense IQP, $n=56$ (96 probabilities)	backend 86 s (8% err) vs cliffft exact 398 s MitM 85 s, QuiZX 2188 s (both exact)	Quokka# (timeout), aer (0/96 in 455 s)
dense IQP, $n=72$	backend 267 s, 1.9 GB	every exact method (~1 TB)
hidden shift, 96 qubits	cliffft μs vs backend 8 s — dispatch picks cliffft	—
magic conveyor, 72 qubits	cliffft 1 ms (block = 2^{12}) vs episodic 66 s; at width 128 (block = 2^{49}) only episodic runs	whole-circuit backend (2^{72} recipes)
cultivation (adaptive)	patch frame 4–16× vs per-term engine	aer (rejects dynamic circuits)

No single winner — that is the point: the dispatcher picks per program, before running.

A · Where it wins, where it doesn't

Small peak rank

clifft is faster *and* exact — keep it. (Most circuits, however wide, compile here.)

Large peak rank, sparse magic

The niche: the only feasible method, accuracy dial-controlled.

Adaptive, Clifford-heavy

Real protocols win 2–16× — and the standard open-source tool cannot run them at all.

- Also measured where it *loses* (up to $\sim 2\times$ overhead) — the advantage is a measured boundary, not a slogan.

Status: prototype validated; honest benchmarks, natural workloads, external baselines done.

B · The idea: don't run one shot faster — run 4,096 abreast

- One shot's small array **cannot keep a GPU busy**: a stadium choir asked to sing one sentence.
- But workloads are **thousands of shots** of the same program — today run one after another.
- Put $B=4,096$ shots side by side; every gate hits *all of them* in one launch.

CPU today: single file



GPU: all at once



one instruction, B statevectors

Batching doesn't make the GPU faster — it makes the problem big enough for the GPU to be fast.

B · Why it should work (1): shots march in lockstep

- A shot's shape — what runs when, how big the array is — is **identical for every shot**, fixed at compile time by cliff's compiler.
- Per-shot randomness is just a few numbers: **SIMD lanes, one per shot**. The classic “shots diverge” objection *does not apply here*.
- And nobody serves this niche: existing batched simulators **fall back to one-at-a-time** on mid-circuit measurement — cliff's home turf.

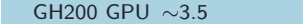
The hardest architectural question resolves in the design's favor before any GPU code is written.

B · Why it should work (2): the speed limit is bandwidth — and GPUs have $7\times$ more

- These kernels do almost no math per byte: **memory speed is the speed limit**, on any machine.
- Measured: 11 CPU cores = only $2.6\times$ one core — the bus, not the cores.
- A GH200 has $\sim 7\times$ the bandwidth $\Rightarrow$ **$5\text{--}10\times$ more shots/s**, honestly bounded.

memory bandwidth (TB/s)

 server CPU ~ 0.5

 GH200 GPU ~ 3.5

big published “ $100\times$ ” numbers use weak baselines — we keep a refuted-claims list and quote the physics.

The claim is deliberately modest — which is why we can trust it.

B · The one risk, and the \$20 experiment that settles it

- Mid-circuit measurement needs the CPU: each crossing costs microseconds — **the same scale as the kernels themselves.**
- But it is paid **once per batch** (nanoseconds per shot), on a link built to make it cheap.
- A faithful microbenchmark — *measurements and round-trips included* — is built, validated, and waiting. **GH200 time: \$2/hour.**

round-trip measured cheap	go: batched backend; first target = replay-style queries (no round-trips at all)
round-trip eats the win	no-go, cleanly — or on-device sampling is the follow-up

Either answer is cheap, quantitative, and final — that is what the microbenchmark is for.

How the two bets fit together

A · Stabilizer rank extends *reach*

circuits cliff cannot hold at all today —
approximate, with a measured accuracy dial

B · GPU batching extends *throughput*

the circuits cliff already runs —
many more shots per second, exact as ever

- Both work *because of* cliff's structure: the frame, the small block, the compile-time schedule.
- Both were reviewed adversarially; every headline number survived a fair baseline.

Next: A — benchmarks complete; write it up. B — the GH200 run, then batched replay queries.